

An edge-deletion matching-split identity for finite
simple graphs, and its instantiation on the prism
graph
(Lean 4 machine-checked claim of record)

Aurora0134*

September 29, 2026

Abstract

For any finite simple graph G with a decidable adjacency relation and any edge ab of G , the number of matchings of G splits as the number of matchings of G with the edge ab deleted, plus the number of matchings of the subgraph induced on the complement of the two endpoints: every matching either avoids ab or contains it, and a matching containing ab uses no other edge at either endpoint. This classical counting device of the matching polynomial / Hosoya-index literature has, to the knowledge reachable by this pipeline’s novelty search, no Lean formalisation: `mathlib` (v4.34.0) has matching predicates but no matching-count API at all, and no same-form statement was found in the public Lean ecosystem. We report a complete, machine-checked Lean 4 formalisation of the identity as a general lemma about arbitrary graphs, together with its instantiation on the prism graph $C_n \square P_2$ (delete one spoke, endpoints kept) where the induced factor is a path P_{n-1} , giving $\mu(C_n \square P_2) = \mu(\text{prism minus one spoke}) + \lfloor \dots \rfloor$ as the first structural step of the pipeline’s defective-prism program. Both theorems compile with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. The identity is asserted as a *first Lean formalisation of a known classical tool*, never as new mathematics. This note is a priority claim of record, not a full paper.

*Contact: via the GitHub account Aurora0134; ORCID placeholder — to be filled in by the author before any upload. No personal information is carried in this note.

1 Introduction

One lemma about arbitrary finite simple graphs and one instantiation on the prism graph are reported here. The lemma is the standard edge-deletion split of matching counts: for a graph G and an edge $e = ab$,

$$\mu(G) = \mu(G - e) + \mu(G - \{a, b\}),$$

where μ counts all matchings (the empty one included) and $G - \{a, b\}$ denotes the graph induced on the complement of $\{a, b\}$. It follows from the partition of matchings into those avoiding and those containing e ; a matching containing e cannot use any other edge incident to a or b , and after removing e the remainder is exactly a matching of the induced graph on the remaining vertices. In the matching polynomial and Hosoya-index literature this split is the classical per-edge device behind essentially every computed matching count [3]; the prism graph’s own OEIS entry A102080 [5] is indexed under the recurrence it feeds. The pipeline’s novelty search (Section 5) found no Lean formalisation of the identity anywhere in the public ecosystem, and mathlib has no matching-count API to state it against [2]; the identity is therefore claimed at the tier *first formalisation (known unformalised)*, with no claim of new mathematics.

The proofs were produced by the AI4Math pipeline, in which LLM workers generate statements and proof scripts and the Lean 4 kernel is the sole adjudicator at every gate: statements are frozen by hash before proving, and a separate audit re-computes the symmetric difference, scans the whole file for `sorry/admit` including comments, and re-prints the axiom dependencies [1]. Verification strength is stated below verbatim. This note is a priority claim of record deposited so that a dated, citable public record exists ahead of the narrative paper of the parent program (the defective-prism matching slices); that paper is prepared separately and will reference this deposit. No literature review is attempted here.

2 Statement

Throughout, graphs are finite simple graphs on a decidable type of vertices; `hosoya` G denotes $|M(G)|$, the number of matchings of G , defined as the cardinality of the set of edge-subsets whose elements pairwise have disjoint endpoint sets (the empty matching included). The prism graph is `prism` $n = (C_n \square P_2)$ on vertices `Fin` $n \times \text{Fin } 2$; `prismDelSpoke` n is the prism with the spoke edge $((0, 0), (0, 1))$ deleted, endpoints kept (requirement $3 \leq n$ so the deletion is well typed).

Lemma 1 (edge-deletion matching-split identity; general form). Let G be a finite simple graph with decidable adjacency, and let a, b be adjacent vertices. Then

$$\text{hosoya } G = \text{hosoya } (G.\text{deleteEdges } \{ab\}) + \text{hosoya } (G.\text{induce } (\text{all vertices except } a, b)).$$

Listing 1: Frozen base definitions, verbatim lines 45–62 of `formalized/statement.lean` (sha256 d186ff16): `edgeSupp`, `hosoya`, `prism`, `prismDelSpoke` and their decidability instances.

```

1 def edgeSupp {V : Type*} [DecidableEq V] [Fintype V] (e : Sym2 V)
  : Finset V :=
2   (univ : Finset V).filter fun v => v ∈ e
3
4 def hosoya (G : SimpleGraph V) [Fintype V] [DecidableEq V] [
  DecidableRel G.Adj] : ℕ :=
5   ((G.edgeFinset.powerset).filter fun s =>
6     (∀ e ∈ s, ∀ f ∈ s, e ≠ f → Disjoint (edgeSupp e) (edgeSupp f)
7     )))
8
9 def prism (n : ℕ) : SimpleGraph (Fin n × Fin 2) := cycleGraph n
10  □ pathGraph 2
11
12 instance (n : ℕ) : DecidableRel (prism n).Adj :=
13   show DecidableRel (cycleGraph n □ pathGraph 2).Adj from
14   inferInstance
15
16 def prismDelSpoke (n : ℕ) (hn : 3 ≤ n) : SimpleGraph (Fin n × Fin
17  2) :=
18   (prism n).deleteEdges {Sym2.mk ((⟨0, by omega⟩, (0 : Fin 2))) ((⟨
19     0, by omega⟩, (1 : Fin 2)))}
20
21 instance (n : ℕ) (hn : 3 ≤ n) : DecidableRel (prismDelSpoke n hn)
22   .Adj :=
23   show DecidableRel ((prism n).deleteEdges
24     {Sym2.mk ((⟨0, by omega⟩, (0 : Fin 2))) ((⟨0, by omega⟩, (1 :
25       Fin 2)))})
26   .Adj from inferInstance

```

Listing 2: General lemma, verbatim head of `attempts/phase0/r2/split-identity.lean` lines 215–259 (declaration and first half of the proof).

```

1 theorem hosoya_deleteEdge_split_pts {V : Type*} [Fintype V] [
  DecidableEq V]

```

```

2   (G : SimpleGraph V) [DecidableRel G.Adj]
3   (a b : V) (huv : G.Adj a b) :
4   hosoya G = hosoya (G.deleteEdges {Sym2.mk a b}) +
5     hosoya (SimpleGraph.induce (Set.univ \ {a, b} : Set V) G) :=
6   by
7   have he : Sym2.mk a b ∈ G.edgeFinset := by
8     rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet];
9     exact huv
10  have hA : ((G.edgeFinset.powerset).filter fun s =>
11    (∀ e₁ ∈ s, ∀ f ∈ s, e₁ ≠ f → Disjoint (edgeSupp e₁) (
12      edgeSupp f)) ∧ Sym2.mk a b ∈ s).card
13    = ((G.edgeFinset.powerset).filter fun u =>
14      (∀ a₁ ∈ u, ∀ b₁ ∈ u, a₁ ≠ b₁ → Disjoint (edgeSupp a₁) (
15        edgeSupp b₁)) ∧
16        ∀ f ∈ u, Disjoint (edgeSupp f) (edgeSupp (Sym2.mk a b))))
17    .card := by
18  apply Finset.card_bij (fun s _ => s.erase (Sym2.mk a b))
19  · intro s hs
20    rw [Finset.mem_filter, Finset.mem_powerset] at hs
21    obtain ⟨hsE, hsP, hse⟩ := hs
22    rw [Finset.mem_filter, Finset.mem_powerset]
23    refine ⟨(Finset.erase_subset _ s).trans hsE, ?_, ?_⟩
24    · intro a₁ ha b₁ hb hab
25      exact hsP a₁ (Finset.mem_of_mem_erase ha) b₁ (Finset.
26        mem_of_mem_erase hb) hab
27    · intro f hf
28      exact hsP f (Finset.mem_of_mem_erase hf) (Sym2.mk a b) hse
29        (Finset.ne_of_mem_erase hf)
30  · intro s₁ hs₁ s₂ hs₂ hh
31    rw [Finset.mem_filter] at hs₁ hs₂
32    rw [← Finset.insert_erase hs₁.2.2, ← Finset.insert_erase
33      hs₂.2.2, hh]
34  · intro u hu
35    rw [Finset.mem_filter, Finset.mem_powerset] at hu
36    obtain ⟨huE, huP, hdj⟩ := hu
37    have heu : Sym2.mk a b ∉ u := by
38      intro heu'
39      have h := hdj (Sym2.mk a b) heu'
40      obtain ⟨v, hv⟩ := Finset.card_pos.mp (by rw [
41        edgeSupp_card_two G (Sym2.mk a b) he]; norm_num)
42      exact Finset.disjoint_left.mp h hv hv
43    refine ⟨insert (Sym2.mk a b) u, ?_, Finset.erase_insert heu⟩
44    rw [Finset.mem_filter, Finset.mem_powerset]
45    refine ⟨Finset.insert_subset he huE, ?_, Finset.
46      mem_insert_self _ _⟩

```

```

37   intro a1 ha b1 hb hab
38   rw [Finset.mem_insert] at ha hb
39   rcases ha with rfl | ha
40   · rcases hb with rfl | hb
41     · exact absurd rfl hab
42     · exact (hdj b1 hb).symm
43   · rcases hb with rfl | hb
44     · exact hdj a1 ha
45     · exact huP a1 ha b1 hb hab

```

Theorem 1 (instantiation on the prism graph). *For every n with $3 \leq n$, the matching counts of the n -prism split along the spoke edge $((0,0), (0,1))$:*

$$\begin{aligned} \text{hosoya}(\text{prism } n) &= \text{hosoya}(\text{prismDelSpoke } n) \\ &\quad + \text{hosoya}(\text{prism induced on the two points' complement}). \end{aligned}$$

The induced factor is the path P_{n-1} (numerically the ladder values 8, 21, ...); its identification with P_{n-1} is not kernel-proved here and is not claimed.

The complete final file is reproduced verbatim in Section 3 (whole file, no excerpting); it also ships in this deposit under `proofs/`. The frozen statement file `formalized/statement.lean` of the underlying task also contains the program’s two main bridge theorems (defective-prism matchings equal the recurrence-defined sequences) with `sorry` placeholders; they are *not* results of this deposit and are mentioned only to delimit its scope. The statement excerpt above stops at the base definitions, so no `sorry` line enters any listing.

3 Proof

The proof of Lemma 1 partitions the power set of the edge finset by “contains ab ” / “avoids ab ” and counts each side: the avoiding side is the matching filter on the edge finset with the edge removed (a definitional rewriting of `deleteEdges`); the containing side is mapped bijectively, via `Finset.card_bij`, to the matching filter of the induced graph on the two-point complement, with each e -free matching of G lifted edge by edge across the vertex-type change using `Sym2.map` and its disjointness/injectivity transport lemmas. The instantiation Theorem 1 applies the general lemma to $G = \text{prism } n$ with ab the spoke, discharging the member adjacency through the box-product adjacency rule. The complete final file follows verbatim (429 lines, no excerpting); it also ships in this deposit under `proofs/`.

Listing 3: Complete proof file of Lemma 1 and Theorem 1, verbatim lines 1–429 (whole file) of `attempts/phase0/r2/split-identity.lean` (sha256 d2efac51).

```

1  import Mathlib
2
3  open SimpleGraph Finset
4
5  /-!
6  # Phase 0 删边分割恒等式 · 完全证明 (2026-09-29, 零占位、严格编译通过)
7
8  provenance:
9  - 基座代码逐字承冻结快照 `formalized/statement.lean` (宪条 2, 只许拷贝不许
10    改)。
11  - 「已验证块 A-D」逐字承 `attempts/phase0/r1/scratch-probe1.lean` 与
12    `scratch-probe2.lean` (军政部 headless 工人 r1b 留下, 主代理 `lake env
13    lean`
14    实测两文件均 exit 0 / 0 error, 2026-09-29 15:2x); 本文件把两探针的
15    example
16    转成可引用的 theorem (证明体逐字未动; `edgeSupp_card_two` 前加 `include
17    G he`
18    一处——Lean 4 section 变量只按声明签名收绑定件, 证明体里单独用要 include
19    )。
20  - 一般装置引理 `hosoya_deleteEdge_split_pts` (两点集形态): 数学骨架承工人
21    r2 在
22    `r2-scratch.lean` 中证明的 edgeSp 形态版 (主代理实测编译通过), 由主代理
23    适配为
24    两点集形态——原因: `Set.univ \ ↑(edgeSupp ...)` 经 Finset coercion, TC
25    合成
26    `Set.univ.fintypeDiffLeft` 实例, 与变量集形态的通用 `Subtype.fintype` 实
27    例
28    非 defeq (实测 8M 心跳 type mismatch), 实例化时 rw/exact 全线失配;
29    两点集字面量与冻结陈述同一条实例合成路径 (pts 探针实证 exit 0)。
30  - 目标定理 `hosoya_prism_split_spoke`: statement 逐字冻结, 实例化走两点集
31    引理
32    + `boxProd_adj` 邻接路线 (承工人 P2 探针)。
33  -/
34
35  -- ===== 基座代码 (逐字承冻结快照 formalized/statement.lean, 只许拷贝不许
36    改) =====
37
38  instance (n : ℕ) : DecidableRel (pathGraph n).Adj := fun _ _ =>
39    decidable_of_iff _ pathGraph_adj.symm
40
41  instance {α β : Type*} [DecidableEq α] [DecidableEq β] (G : SimpleGraph
42    α) (H : SimpleGraph β)
43    [DecidableRel G.Adj] [DecidableRel H.Adj] : DecidableRel (G □ H).Adj
44    := fun _ _ =>
45    decidable_of_iff _ boxProd_adj.symm

```

```

33
34 def edgeSupp {V : Type*} [DecidableEq V] [Fintype V] (e : Sym2 V) :
    Finset V :=
35   (univ : Finset V).filter fun v => v ∈ e
36
37 def hosoya (G : SimpleGraph V) [Fintype V] [DecidableEq V] [DecidableRel
    G.Adj] : ℕ :=
38   ((G.edgeFinset.powerset).filter fun s =>
39     (∀ e ∈ s, ∀ f ∈ s, e ≠ f → Disjoint (edgeSupp e) (edgeSupp f))).
    card
40
41 def prism (n : ℕ) : SimpleGraph (Fin n × Fin 2) := cycleGraph n □
    pathGraph 2
42
43 instance (n : ℕ) : DecidableRel (prism n).Adj :=
44   show DecidableRel (cycleGraph n □ pathGraph 2).Adj from inferInstance
45
46 def prismDelSpoke (n : ℕ) (hn : 3 ≤ n) : SimpleGraph (Fin n × Fin 2) :=
47   (prism n).deleteEdges {Sym2.mk ((⟨0, by omega⟩, (0 : Fin 2))) ((⟨0, by
    omega⟩, (1 : Fin 2)))}
48
49 instance (n : ℕ) (hn : 3 ≤ n) : DecidableRel (prismDelSpoke n hn).Adj :=
50   show DecidableRel ((prism n).deleteEdges
51     {Sym2.mk ((⟨0, by omega⟩, (0 : Fin 2))) ((⟨0, by omega⟩, (1 : Fin 2)))
52     }).Adj from inferInstance
53
54 -- ===== 已验证块 A (承 scratch-probe1.lean, rfl 展开层) =====
55 theorem hosoya_def {V : Type*} [DecidableEq V] [Fintype V] (G :
    SimpleGraph V)
56   [DecidableRel G.Adj] :
57   hosoya G = ((G.edgeFinset.powerset).filter fun s =>
58     (∀ e ∈ s, ∀ f ∈ s, e ≠ f → Disjoint (edgeSupp e) (edgeSupp f))).
    card := rfl
59
60 -- ===== 已验证块 B (承 scratch-probe1.lean P2/P4, 棱柱 spoke 事实) =====
61
62 theorem spoke_mem_edgeFinset (n : ℕ) (hn : 3 ≤ n) :
63   Sym2.mk ((⟨0, by omega⟩ : Fin n), (0 : Fin 2))
64     ((⟨0, by omega⟩ : Fin n), (1 : Fin 2)) ∈ (prism n).edgeFinset := by
65   rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet]
66   show (cycleGraph n □ pathGraph 2).Adj ((⟨0, by omega⟩, (0 : Fin 2)) ((⟨0,
    by omega⟩, (1 : Fin 2)))
67   rw [boxProd_adj]
68   refine Or.inr ⟨?, rfl⟩
69   show (pathGraph 2).Adj (0 : Fin 2) (1 : Fin 2)
70   decide
71
72 theorem edgeSupp_spoke (n : ℕ) (hn : 3 ≤ n) :

```

```

73   (↑(edgeSupp (Sym2.mk ((⟨0, by omega⟩ : Fin n), (0 : Fin 2))
74     ((⟨0, by omega⟩ : Fin n), (1 : Fin 2)))) : Set (Fin n × Fin 2)) =
75   {((⟨0, by omega⟩ : Fin n), (0 : Fin 2)), ((⟨0, by omega⟩ : Fin n), (1
    : Fin 2))} := by
76   ext v
77   simp [edgeSupp, Sym2.mem_iff]
78
79   -- ===== 已验证块 C (承 scratch-probe1.lean P5/P6, 不含 e 半 + 分区计数)
80   =====
81   theorem matching_filter_not_mem (V : Type*) [Fintype V] [DecidableEq V] (
82     G : SimpleGraph V)
83     [DecidableRel G.Adj] (e : Sym2 V) :
84     (G.edgeFinset.powerset.filter fun s =>
85       (∀ e₁ ∈ s, ∀ f ∈ s, e₁ ≠ f → Disjoint (edgeSupp e₁) (edgeSupp f
86         )) ∧ e ∉ s) =
87     ((G.edgeFinset \ {e}).powerset).filter fun s =>
88       (∀ e₁ ∈ s, ∀ f ∈ s, e₁ ≠ f → Disjoint (edgeSupp e₁) (edgeSupp f
89         )) := by
90   ext s
91   simp only [Finset.mem_powerset, Finset.mem_filter]
92   constructor
93   · rintro ⟨hs, hP, hne⟩
94     refine ⟨fun x hx => ?_, hP⟩
95     rw [Finset.mem_sdiff, Finset.mem_singleton]
96     exact ⟨hs hx, fun h => hne (h ▶ hx)⟩
97   · rintro ⟨hs, hP⟩
98     refine ⟨fun x hx => (Finset.mem_sdiff.mp (hs hx)).1, hP, fun h => ?_⟩
99     exact (Finset.mem_sdiff.mp (hs h)).2 (Finset.mem_singleton_self e)
100
101   theorem hosoya_partition (V : Type*) [Fintype V] [DecidableEq V] (G :
102     SimpleGraph V)
103     [DecidableRel G.Adj] (e : Sym2 V) :
104     hosoya G = ((G.edgeFinset.powerset.filter fun s =>
105       (∀ e₁ ∈ s, ∀ f ∈ s, e₁ ≠ f → Disjoint (edgeSupp e₁) (edgeSupp f
106         )) ∧ e ∉ s).card +
107       (G.edgeFinset.powerset.filter fun s =>
108         (∀ e₁ ∈ s, ∀ f ∈ s, e₁ ≠ f → Disjoint (edgeSupp e₁) (edgeSupp f
109           )) ∧ e ∈ s).card) := by
110   have h1 := Finset.card_filter_add_card_filter_not
111     (s := G.edgeFinset.powerset.filter fun s =>
112       (∀ e₁ ∈ s, ∀ f ∈ s, e₁ ≠ f → Disjoint (edgeSupp e₁) (edgeSupp f))
113     ) (fun s => e ∉ s)
114   rw [hosoya_def, ← h1]
115   congr 1
116   · rw [Finset.filter_filter]
117   · rw [Finset.filter_filter]
118     congr 1
119   ext s

```

```

113     simp only [not_not]
114
115 -- ===== 已验证块 D (承 scratch-probe2.lean, Sym2 跨顶点型搬运四件) =====
116
117 theorem edgeSupp_mem {V : Type*} [DecidableEq V] [Fintype V] (e : Sym2 V)
118   (v : V) :
119     v ∈ edgeSupp e ↔ v ∈ e := by simp [edgeSupp]
120
121 section Transport
122
123 variable {V : Type*} [Fintype V] [DecidableEq V] (G : SimpleGraph V) [
124   DecidableRel G.Adj]
125 (e : Sym2 V) (he : e ∈ G.edgeFinset)
126
127 theorem edgeSupp_map_subtype (g : Sym2 ↑(↑((univ : Finset V) \ edgeSupp e)
128   : Set V)) :
129   (edgeSupp g).map (Function.Embedding.subtype
130     (· ∈ (↑((univ : Finset V) \ edgeSupp e) : Set V))) =
131   edgeSupp (Sym2.map (Subtype.val) g) := by
132     ext v
133     rw [Finset.mem_map]
134     constructor
135     · rintro ⟨u, hu, huv⟩
136       rw [edgeSupp_mem] at hu
137       rw [edgeSupp_mem]
138       exact Sym2.mem_map.mpr ⟨u, hu, huv⟩
139     · intro hv
140       rw [edgeSupp_mem] at hv
141       obtain ⟨u, hu, huv⟩ := Sym2.mem_map.mp hv
142       exact ⟨u, by rw [edgeSupp_mem]; exact hu, huv⟩
143
144 theorem disjoint_map_subtype (g₁ g₂ : Sym2 ↑(↑((univ : Finset V) \
145   edgeSupp e) : Set V)) :
146   Disjoint (edgeSupp g₁) (edgeSupp g₂) ↔
147   Disjoint (edgeSupp (Sym2.map (Subtype.val) g₁)) (edgeSupp (Sym2.map (
148     Subtype.val) g₂)) := by
149     have L1 : ∀ g : Sym2 ↑(↑((univ : Finset V) \ edgeSupp e) : Set V),
150       (edgeSupp g).map (Function.Embedding.subtype
151         (· ∈ (↑((univ : Finset V) \ edgeSupp e) : Set V))) =
152       edgeSupp (Sym2.map (Subtype.val) g) := by
153       intro g
154       ext v
155       rw [Finset.mem_map]
156       constructor
157       · rintro ⟨u, hu, huv⟩
158         rw [edgeSupp_mem] at hu
159         rw [edgeSupp_mem]
160         exact Sym2.mem_map.mpr ⟨u, hu, huv⟩
161       · intro hv

```

```

157     rw [edgeSupp_mem] at hv
158     obtain ⟨u, hu, huv⟩ := Sym2.mem_map.mp hv
159     exact ⟨u, by rw [edgeSupp_mem]; exact hu, huv⟩
160   rw [← L1 g₁, ← L1 g₂]
161   constructor
162   · intro h
163     rw [Finset.disjoint_left] at h ⊢
164     intro v hv hv'
165     rw [Finset.mem_map] at hv hv'
166     obtain ⟨u, hu, huv⟩ := hv
167     obtain ⟨u', hu', huv'⟩ := hv'
168     have huu : u = u' := (Function.Embedding.subtype _).injective (huv.
      trans huv'.symm)
169     exact h hu (huu ► hu')
170   · intro h
171     rw [Finset.disjoint_left] at h ⊢
172     intro u hu hu'
173     exact h (Finset.mem_map.mpr ⟨u, hu, rfl⟩) (Finset.mem_map.mpr ⟨u, hu',
      rfl⟩)
174
175   theorem mapped_ne_edge (g : Sym2 ↑(↑((univ : Finset V) \ edgeSupp e) : Set
      V)) :
176     Sym2.map (Subtype.val) g ≠ e := by
177     intro hge
178     have hex : ∃ u : V, u ∈ e := by
179       induction e using Sym2.ind with
180       | _ a b => exact ⟨a, Sym2.mem_iff.mpr (Or.inl rfl)⟩
181     obtain ⟨u, hu⟩ := hex
182     have huE : u ∈ edgeSupp e := Finset.mem_filter.mpr ⟨Finset.mem_univ u,
      hu⟩
183     have hu2 : u ∈ edgeSupp (Sym2.map (Subtype.val) g) := hge.symm ► huE
184     rw [edgeSupp_mem] at hu2
185     obtain ⟨w, hwg, hwu⟩ := Sym2.mem_map.mp hu2
186     rw [← hwu] at huE
187     have hwT : (↑w : V) ∈ (↑((univ : Finset V) \ edgeSupp e) : Set V) := w
      .2
188     rw [Finset.mem_coe, Finset.mem_sdiff] at hwT
189     exact hwT.2 huE
190
191   include G he
192   theorem edgeSupp_card_two : (edgeSupp e).card = 2 := by
193     rw [SimpleGraph.mem_edgeFinset] at he
194     induction e using Sym2.ind with
195     | _ a b =>
196       rw [SimpleGraph.mem_edgeSet] at he
197       have hab : a ≠ b := G.ne_of_adj he
198       rw [Finset.card_eq_two]
199       refine ⟨a, b, hab, ?_⟩
200     ext v

```

```

201     simp [edgeSupp, Sym2.mem_iff]
202
203 end Transport
204
205 -- ===== 一般装置引理（删边分割恒等式 · 两点集形态） =====
206
207 /-- 一般装置引理（删边分割恒等式，两点集形态）：对任意图 `G` 与边 `a ~ b`，
208 匹配集按「含 `s(a, b)` / 不含 `s(a, b)`」二分——不含者 = `G.deleteEdges {s
    (a, b)}`
209 的匹配数（已验证块  $C$ ），含者 = 删 `a, b` 两 endpoint 后诱导子图的匹配数（ $B$  侧：
210  `Finset.card_bij` 双豫 +  $Sym2$  跨型搬运）。
211 形态说明：诱导集用 `Set.univ \ {a, b}`（两点集字面量）而非 `Set.univ \ ↑(
    edgeSupp ...)``——
212 后者经 Finset coercion,  $TC$  合成的是 `Set.univ.fintypeDiffLeft` 实例，与变量
    集形态的
213 通用 `Subtype.fintype` 实例非 defeq（实测 SM 心跳 type mismatch），会导致实
    例化时
214 rw/exact 全线失配；两点集字面量形态与冻结陈述同一条实例合成路径（pts 探针实
    证）。 -/
215 theorem hosoya_deleteEdge_split_pts {V : Type*} [Fintype V] [DecidableEq
    V]
216   (G : SimpleGraph V) [DecidableRel G.Adj]
217   (a b : V) (huv : G.Adj a b) :
218   hosoya G = hosoya (G.deleteEdges {Sym2.mk a b}) +
219     hosoya (SimpleGraph.induce (Set.univ \ {a, b} : Set V) G) := by
220 have he : Sym2.mk a b ∈ G.edgeFinset := by
221   rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet]; exact huv
222 have hA : ((G.edgeFinset.powerset).filter fun s =>
223   (∀ e₁ ∈ s, ∀ f ∈ s, e₁ ≠ f → Disjoint (edgeSupp e₁) (edgeSupp f
    )) ∧ Sym2.mk a b ∈ s).card
224   = ((G.edgeFinset.powerset).filter fun u =>
225     (∀ a₁ ∈ u, ∀ b₁ ∈ u, a₁ ≠ b₁ → Disjoint (edgeSupp a₁) (edgeSupp
    b₁))) ∧
226     ∀ f ∈ u, Disjoint (edgeSupp f) (edgeSupp (Sym2.mk a b))).card
    := by
227 apply Finset.card_bij (fun s _ => s.erase (Sym2.mk a b))
228 · intro s hs
229   rw [Finset.mem_filter, Finset.mem_powerset] at hs
230   obtain ⟨hsE, hsP, hse⟩ := hs
231   rw [Finset.mem_filter, Finset.mem_powerset]
232   refine ⟨(Finset.erase_subset _ s).trans hsE, ?_, ?_⟩
233   · intro a₁ ha b₁ hb hab
234     exact hsP a₁ (Finset.mem_of_mem_erase ha) b₁ (Finset.
    mem_of_mem_erase hb) hab
235   · intro f hf
236     exact hsP f (Finset.mem_of_mem_erase hf) (Sym2.mk a b) hse (
    Finset.ne_of_mem_erase hf)
237   · intro s₁ hs₁ s₂ hs₂ hh
238     rw [Finset.mem_filter] at hs₁ hs₂

```

```

239   rw [← Finset.insert_erase hs1.2.2, ← Finset.insert_erase hs2.2.2,
      hh]
240   · intro u hu
241     rw [Finset.mem_filter, Finset.mem_powerset] at hu
242     obtain ⟨huE, huP, hdj⟩ := hu
243     have heu : Sym2.mk a b ∉ u := by
244       intro heu'
245       have h := hdj (Sym2.mk a b) heu'
246       obtain ⟨v, hv⟩ := Finset.card_pos.mp (by rw [edgeSupp_card_two G (
Sym2.mk a b) he]; norm_num)
247       exact Finset.disjoint_left.mp h hv hv
248       refine ⟨insert (Sym2.mk a b) u, ?_, Finset.erase_insert heu⟩
249       rw [Finset.mem_filter, Finset.mem_powerset]
250       refine ⟨Finset.insert_subset he huE, ?_, Finset.mem_insert_self _ _⟩
251       intro a1 ha b1 hb hab
252       rw [Finset.mem_insert] at ha hb
253       rcases ha with rfl | ha
254       · rcases hb with rfl | hb
255         · exact absurd rfl hab
256         · exact (hdj b1 hb).symm
257       · rcases hb with rfl | hb
258         · exact hdj a1 ha
259         · exact huP a1 ha b1 hb hab
260   have hB : ((G.edgeFinset.powerset).filter fun u =>
261     (∀ a1 ∈ u, ∀ b1 ∈ u, a1 ≠ b1 → Disjoint (edgeSupp a1) (edgeSupp
b1))) ∧
262     (∀ f ∈ u, Disjoint (edgeSupp f) (edgeSupp (Sym2.mk a b))).card
263     = (((SimpleGraph.induce (Set.univ \ {a, b} : Set V) G).edgeFinset.
powerset).filter
264       fun t => (∀ a1 ∈ t, ∀ b1 ∈ t, a1 ≠ b1 → Disjoint (edgeSupp a1)
(edgeSupp b1))).card := by
265   have hTS : ∀ v : V, v ∉ edgeSupp (Sym2.mk a b) → v ∈ (Set.univ \ {a,
b} : Set V) := by
266     intro v hv
267     refine ⟨Set.mem_univ v, ?_⟩
268     intro hmem
269     apply hv
270     rw [edgeSupp_mem, Sym2.mem_iff]
271     simp using hmem
272   have map_inj : Function.Injective
273     (Sym2.map (Subtype.val : |(Set.univ \ {a, b} : Set V) → V)) := by
274     intro x y hxy
275     induction x using Sym2.ind with
276     | _ x1 y1 =>
277       induction y using Sym2.ind with
278       | _ x2 y2 =>
279         rw [Sym2.map_mk, Sym2.map_mk, Sym2.eq_iff] at hxy
280         rw [Sym2.eq_iff]
281         rcases hxy with ⟨h1, h2⟩ | ⟨h1, h2⟩

```

```

282     · exact Or.inl ⟨Subtype.ext h1, Subtype.ext h2⟩
283     · exact Or.inr ⟨Subtype.ext h1, Subtype.ext h2⟩
284 have hlift : ∀ f : Sym2 V, f ∈ G.edgeFinset → Disjoint (edgeSupp f)
  (edgeSupp (Sym2.mk a b)) →
285   ∃ g : Sym2 ↑(Set.univ \ {a, b} : Set V),
286     Sym2.map Subtype.val g = f ∧
287     g ∈ (SimpleGraph.induce (Set.univ \ {a, b} : Set V) G).
edgeFinset := by
288   intro f hfE hfdj
289   induction f using Sym2.ind with
290   | _ x y =>
291     rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet] at hfE
292     have hxE : x ∉ edgeSupp (Sym2.mk a b) := by
293       intro ha
294       exact Finset.disjoint_left.mp hfdj
295       (by rw [edgeSupp_mem]; exact Sym2.mem_iff.mpr (Or.inl rfl))
ha
296     have hyE : y ∉ edgeSupp (Sym2.mk a b) := by
297       intro hb
298       exact Finset.disjoint_left.mp hfdj
299       (by rw [edgeSupp_mem]; exact Sym2.mem_iff.mpr (Or.inr rfl))
hb
300     refine ⟨s(⟨(x, hTS x hxE) : ↑(Set.univ \ {a, b} : Set V)),
301       ⟨(y, hTS y hyE) : ↑(Set.univ \ {a, b} : Set V)⟩), ?_, ?_⟩
302     · rw [Sym2.map_mk]
303     · rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet]
304       exact SimpleGraph.induce_adj.mpr hfE
305 symm
306 apply Finset.card_bij (fun t _ => t.image (Sym2.map (Subtype.val :
307   ↑(Set.univ \ {a, b} : Set V) → V)))
308 · intro t ht
309   rw [Finset.mem_filter, Finset.mem_powerset] at ht
310   obtain ⟨htE, htP⟩ := ht
311   rw [Finset.mem_filter, Finset.mem_powerset]
312   refine ⟨?_, ?_, ?_⟩
313   · intro y hy
314     obtain ⟨g, hg, rfl⟩ := Finset.mem_image.mp hy
315     have hgE := htE hg
316     induction g using Sym2.ind with
317     | _ x y =>
318       rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet] at hgE
319       rw [Sym2.map_mk, SimpleGraph.mem_edgeFinset, SimpleGraph.
mem_edgeSet]
320       exact SimpleGraph.induce_adj.mp hgE
321     · intro x hx y₁ hy hxy
322       obtain ⟨g₁, hg₁, rfl⟩ := Finset.mem_image.mp hx
323       obtain ⟨g₂, hg₂, rfl⟩ := Finset.mem_image.mp hy
324       have hgg : g₁ ≠ g₂ := fun h => hxy (by rw [h])
325       have hdj := htP g₁ hg₁ g₂ hg₂ hgg

```

```

326   rw [Finset.disjoint_left] at hdj ⊢
327   intro v hv1 hv2
328   rw [edgeSupp_mem] at hv1 hv2
329   obtain ⟨u1, hu1, huv1⟩ := Sym2.mem_map.mp hv1
330   obtain ⟨u2, hu2, huv2⟩ := Sym2.mem_map.mp hv2
331   have huu : u1 = u2 := Subtype.ext (huv1.trans huv2.symm)
332   exact hdj ((edgeSupp_mem g1 u1).mpr hu1) ((edgeSupp_mem g2 u1).mpr
(huu ▸ hu2))
333   · intro y hy
334     obtain ⟨g, hg, rfl⟩ := Finset.mem_image.mp hy
335     rw [Finset.disjoint_left]
336     intro v hv hve
337     rw [edgeSupp_mem] at hv
338     obtain ⟨u, hu, huv⟩ := Sym2.mem_map.mp hv
339     rw [← huv] at hve
340     have huS := (Set.mem_sdiff _).mp u.2
341     have hmem : (u.1 : V) ∈ ({a, b} : Set V) := by
342       have h1 : u.1 ∈ Sym2.mk a b := (edgeSupp_mem _ _).mp hve
343       have h2 : u.1 = a ∨ u.1 = b := (Sym2.mem_iff).mp h1
344       simpa using h2
345     exact huS.2 hmem
346   · intro t1 h1 t2 h2 hh
347     ext g
348     constructor
349     · intro hg
350       have hm : Sym2.map Subtype.val g ∈ t2.image (Sym2.map Subtype.val
) := by
351         rw [← hh]
352         exact Finset.mem_image.mpr ⟨g, hg, rfl⟩
353         obtain ⟨g', hg', hgg'⟩ := Finset.mem_image.mp hm
354         exact (map_inj hgg') ▸ hg'
355     · intro hg
356       have hm : Sym2.map Subtype.val g ∈ t1.image (Sym2.map Subtype.val
) := by
357         rw [hh]
358         exact Finset.mem_image.mpr ⟨g, hg, rfl⟩
359         obtain ⟨g', hg', hgg'⟩ := Finset.mem_image.mp hm
360         exact (map_inj hgg') ▸ hg'
361   · intro u hu
362     rw [Finset.mem_filter, Finset.mem_powerset] at hu
363     obtain ⟨huE, huP, hdj⟩ := hu
364     refine ⟨u.attach.image fun x => Classical.choose (hlift x.1 (huE x
.2) (hdj x.1 x.2)), ?_, ?_⟩
365     · rw [Finset.mem_filter, Finset.mem_powerset]
366       constructor
367       · intro g hg
368         obtain ⟨⟨f, hf⟩, -, rfl⟩ := Finset.mem_image.mp hg
369         exact (Classical.choose_spec (hlift f (huE hf) (hdj f hf))).2
370       · intro g1 hg1 g2 hg2 hgg

```

```

371   obtain ⟨⟨f1, hf1⟩, -, rfl⟩ := Finset.mem_image.mp hg1
372   obtain ⟨⟨f2, hf2⟩, -, rfl⟩ := Finset.mem_image.mp hg2
373   have hff : f1 ≠ f2 := by
374     intro h
375     apply hgg
376     subst h
377     rfl
378   have s1 := (Classical.choose_spec (hlift f1 (huE hf1) (hdj f1 hf
1))))).1
379   have s2 := (Classical.choose_spec (hlift f2 (huE hf2) (hdj f2 hf
2))))).1
380   have hdj2 := huP f1 hf1 f2 hf2 hff
381   rw [Finset.disjoint_left] at hdj2 ⊢
382   intro v hv1 hv2
383   rw [edgeSupp_mem] at hv1 hv2
384   exact hdj2
385     ((edgeSupp_mem f1 v).mpr (s1 ► Sym2.mem_map.mpr ⟨v, hv1, rfl⟩
386 ))
387     ((edgeSupp_mem f2 v).mpr (s2 ► Sym2.mem_map.mpr ⟨v, hv2, rfl⟩
388 ))
389   · ext z
390     constructor
391     · intro hz
392       obtain ⟨g, hg, hgZ⟩ := Finset.mem_image.mp hz
393       obtain ⟨⟨f, hf⟩, -, rfl⟩ := Finset.mem_image.mp hg
394       have s := (Classical.choose_spec (hlift f (huE hf) (hdj f hf)))
395       .1
396         rw [s] at hgZ
397         exact hgZ ► hf
398     · intro hz
399       have s := (Classical.choose_spec (hlift z (huE hz) (hdj z hz)))
400       .1
401         exact Finset.mem_image.mpr ⟨_, Finset.mem_image.mpr ⟨⟨z, hz⟩,
402           Finset.mem_attach _ _, rfl⟩, s⟩
403   have h2 : (G.deleteEdges {Sym2.mk a b}).edgeFinset = G.edgeFinset \ {
404     Sym2.mk a b} := by
405     ext f
406     induction f using Sym2.ind with
407     | _ x y =>
408       simp only [Finset.mem_sdiff, Finset.mem_singleton]
409       rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet,
410         SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet, SimpleGraph.
411         deleteEdges_adj,
412         Set.mem_singleton_iff]
413   have hdel : hosoya (G.deleteEdges {Sym2.mk a b}) =
414     ((G.edgeFinset \ {Sym2.mk a b}).powerset.filter fun s =>
415       (∀ e1 ∈ s, ∀ f ∈ s, e1 ≠ f → Disjoint (edgeSupp e1) (edgeSupp f
416 ))) .card := by
417     rw [hosoya_def, h2]

```

```

410 rw [hosoya_partition V G (Sym2.mk a b), matching_filter_not_mem V G (
    Sym2.mk a b), hdel,
411   hosoya_def, hA, hB]
412 congr 1
413
414 -- ===== 目标定理 (statement 逐字冻结; 实例化走两点集形态引理) =====
415
416 theorem hosoya_prism_split_spoke : ∀ (n : ℕ) (hn : 3 ≤ n),
417   hosoya (prism n) =
418     hosoya (prismDelSpoke n hn) +
419     hosoya (SimpleGraph.induce
420       (Set.univ \ {((⟨0, by omega⟩, (0 : Fin 2))), (⟨⟨0, by omega⟩, (1
        : Fin 2))}) : Set (Fin n × Fin 2))
421       (prism n)) := by
422   intro n hn
423   refine hosoya_deleteEdge_split_pts (prism n) ((⟨0, by omega⟩, (0 : Fin
    2)))
424     ((⟨0, by omega⟩, (1 : Fin 2))) ?_
425   show (cycleGraph n □ pathGraph 2).Adj ((⟨0, by omega⟩, (0 : Fin 2)) (⟨0,
    by omega⟩, (1 : Fin 2))
426   rw [boxProd_adj]
427   refine Or.inr ⟨?_, rfl⟩
428   show (pathGraph 2).Adj (0 : Fin 2) (1 : Fin 2)
429   decide

```

4 Verification evidence

- Toolchain: Lean v4.34.0 (leanprover/lean4:v4.34.0), mathlib4 v4.34.0, revision 5ed29652. Reproduction: with that pin, lake env lean on proofs/01-hosoya-split-proved.lean compiles with exit code 0 and empty output.
- Statement integrity: the frozen statement file formalized/statement.lean (124 lines) has sha256

d186ff1651a4383e7e213b42ba4ec8c01
71d9e7004231db2764370cd967ca39e,

and the final file split-identity.lean (429 lines) has sha256

d2efac514083c505b9fa14675e59a8009
054e2063769c263e069d608f656b016.

The theorem head of `hosoya_prism_split_spoke` is byte-identical to the frozen statement (programmatic comparison, zero symmetric difference; see `audit/statement-diff.out`). The base definitions excerpted in Listing 1 are copied verbatim from the frozen statement file.

- Kernel check: compiles with 0 sorry (whole-file scan including comments, case-insensitive, also covering `admit`, `sorryAx`, `native_decide`, `axiom` and `unsafe`: zero hits); `#print` axioms output, verbatim (kept literal; the smaller monospace width is typesetting only):

```
1 'hosoya_deleteEdge_split_pts' depends on axioms: [propext,
    Classical.choice, Quot.sound]
2 'hosoya_prism_split_spoke' depends on axioms: [propext, Classical.
    choice, Quot.sound]
```

Each line is a subset of the whitelist {`propext`, `Quot.sound`, `Classical.choice`}. The strict re-compile emits no error and no warning (stdout empty); the evidence ships as `audit/audit-strict-compile.log` and `audit/audit-axioms-sidecar.log`.

- Grading by the audit department: both proven declarations *complete proof*; lowest grade reached for the case = *complete proof*, with one provenance caveat stated by the audit and carried here: the mathematical skeleton of Lemma 1 was produced by a worker agent in a scratch probe (compiling) and was adapted to the two-point-set form by the main agent to match the frozen statement’s instance-synthesis path; the provenance is recorded in the final file header and in the audit record, and the deposit does not narrate the lemma as an independent single-author pass.
- Audit chain (originals in this deposit under `audit/` unless noted): final adjudication `final-01-hosoya-split.txt` (four checks: statement comparison, strict compile, sorry scan, axiom audit), the statement symmetric-difference record `statement-diff.out`, and this note’s claim check `claim-check.md` — added to `audit/` after the audit department returns its adjudication, and a prerequisite for publication. The semantic-fidelity check (roundtrip re-translation) of the underlying task ran on the same model node as its formalisation; that independence weakness is mitigated, not removed, by kernel adjudication.

5 Novelty evidence

Adjudication copied from the pipeline’s exclusivity review (C9 gate), which ran before the problem was accepted and re-derived the data independently: verdict **no occupying record found (reachable-channel scope)** for both defective-prism matching slices and for the graph-side statement layer; value tier of the parent topic **new sequence** (the mother sequence A102080 [5] is occupied and is not claimed), with the graph-side bridge graded **first formalisation (Lean three-layer zero same-form)**. The identity itself is not new mathematics and is not asserted as such: as a human-maths tool it is classical in the matching-polynomial literature, and the claim is strictly *no prior Lean formalisation was reachable by the review*. Summary by channel; the full query log, including every zero-hit query, is in this deposit at `audit/c9-record.md`.

channel	queries	results
OES (donner, sequence side) [4]	14 strings, 4 positive controls	two slice families in six forms; zero hits; controls hit A102080 only
OES (graph-side increment)	algorithms [2, 4, 9, -1] index-page sweep	4 entries registered, neither slice among them; neither A102080 confirmed
oXiv	9 (donner + increment)	nearest neighbours read full-text (PI-property in graph prism; runtime keyword collision); no matching-count or edge-deletion same-form
oXiv (package’s supplementary sweep, 2026-09-29)	all “Hosoya index” AND all prism; also “Hosoya index” AND also “deleted” / “edge deletion”	0 entries
Control	7	nearest hit Aldred-Plummer (matching extension): structural, not counting; full text saved, recorded
MSR	5 + 1 control	control hit; also queries zero
Graphs code search (public Lean ecosystem)	6	zero same-form; prism/hosoya/matching-count intersections all empty
mathlib ty layer (pinned v4.34.0)	5	matching predicates only; no counting API; no prism definition; no Hosoya word family
Independent review	7 from	overall = confirmed (exists on two non-load-bearing ledger branches)

Two points of the record are part of the evidence. First, the review’s anchors self-proved (the mother string hitting A102080 with the exact name line fixes what “zero hit” means on the sequence channel). Second, the channels are named so a later reader can re-run them; if any channel later returns a same-form Lean formalisation of the edge-deletion splitting identity, the tier above weakens accordingly, and this deposit will be amended by a later version rather than by editing this record.

6 Scope and limitations

Grade and boundary, stated as the audit states them. (a) The deposit proves Lemma 1 (general form) and Theorem 1 (prism instantiation) and nothing else. (b) The parent program’s two bridge theorems (defective-prism matchings equal recurrence-defined sequences) and the six congruence-pattern theorems of the sequence side are *not* results of this deposit; the bridge theorems are stated with **sorry** in the frozen statement file and their proof phases are in progress. (c) Identification of the induced factor of Theorem 1 with the path P_{n-1} is numerically supported but not kernel-proved here. (d) **hosoya** is defined on decidable graphs; the identity is finite-combinatorial and holds with or without decidability, but the lean formulation fixes the decidable case. (e) The novelty record was scoped to the defective-prism slices; the

packager’s supplementary arXiv sweeps of 2026-09-29 (Section 5) extend the zero-reachability evidence to the identity’s own wording, but channels outside the reachable set (paywalled full texts, e.g. Farrell’s ladder papers, and general web search) remain recorded as unverified rather than as evidence. (f) The gate-four sign-off of the underlying task was exercised by the author’s instruction to deposit this claim on 2026-09-29, while the task report file and the department audit record are added to the working repository alongside this deposit; the task’s Phase 1/2 remain in flight and are not part of this deposit. (g) This note carries no literature review and is a deposit record, not a survey; the narrative paper of the parent program is separate.

Data availability

The deposit package for this claim contains the claim note (LaTeX source and PDF), the Lean sources (`proofs/`: frozen statement file, final proof file, `lean-toolchain`), the verification and audit logs, and the full C9 query log with zero-hit entries (`audit/`). Zenodo: the DOI is reserved at upload by the author and is recorded in `metadata/README.md` of this package once created; see `UPLOAD.md` in the source repository. Claim note under CC BY 4.0, code under MIT.

Use of generative AI

(a) AI performed: candidate generation and the exclusivity review, autoformalisation of the definitions and statements, the roundtrip semantic check, proof search and proof-script writing (worker agents plus main-agent form adaptation on Lemma 1 as recorded above), error classification and repair, and the assembly of this note including its verbatim listings. (b) Model, node and budget: the proof phase ran on one node, wire-id `anthropic/stepfun/step-5-preview`, with no cross-node switching, consuming 2 agent-run samples of a permitted 6 for the phase and 3 kernel compiles of a permitted 36 for the task, plus 8 llm-call requests of a permitted 8 (4 successful roundtrip adjudications and 4 transmission failures), copied verbatim from the task ledger. This note consumed 0 pipeline LLM calls against a cap of 4; drafting was mechanical assembly. (c) Final adjudication: all statements and proofs reported here were checked by the Lean 4 kernel: the proof compiles with no `sorry`, and `#print axioms` reports only {`propext`, `Classical.choice`, `Quot.sound`}. (d) The AI system is not an author; the human author reviewed the evidence and is responsible

for all content.

Author contributions

CRediT-style: the human author adjudicated topic selection, statement semantics and the deposit decision; the AI4Math pipeline (an LLM) generated the candidate propositions, the proof searches and the text drafts. The AI system is not listed as an author.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Automated Deduction – CADE-28*, LNCS, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, pp. 367–381, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] C. D. Godsil and I. Gutman. On the theory of the matching polynomial. *Journal of Graph Theory* 5 (1981), pp. 137–144. <https://doi.org/10.1002/jgt.3190050203>
- [4] The OEIS Foundation. *The On-Line Encyclopedia of Integer Sequences*. <https://oeis.org/>
- [5] OEIS Foundation. Number of matchings in the $C_n \times P_2$ (n -prism) graph. *The On-Line Encyclopedia of Integer Sequences*, entry A102080. <https://oeis.org/A102080>